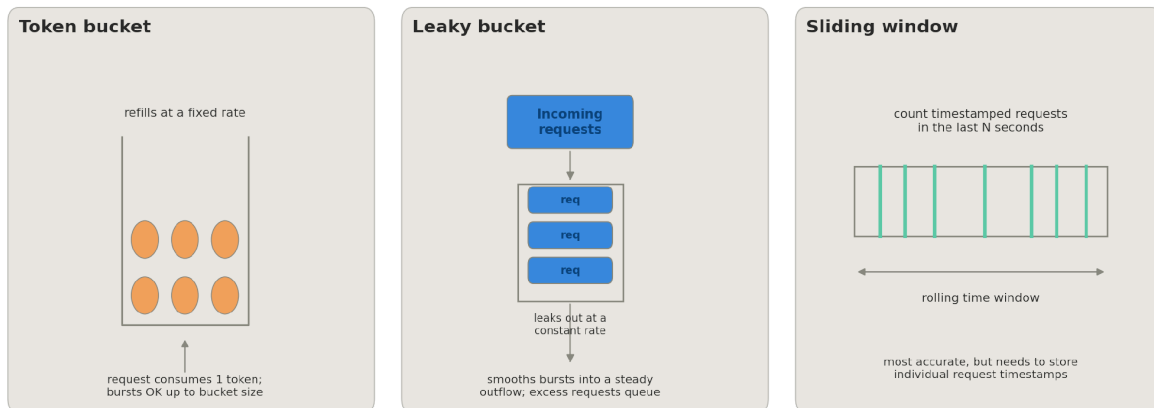


Stage 6 Crash Course: API Design & Service Architecture

With the hard infrastructure problems handled (Stages 1–5), this stage is about how independently-deployed pieces of a system expose themselves to clients and to each other, and stay compatible over time.

1. Rate Limiting

A rate limiter protects a service from being overwhelmed — by abuse, a bug in a client, or just more legitimate traffic than the service can handle — by rejecting requests once a client exceeds an allowed rate.

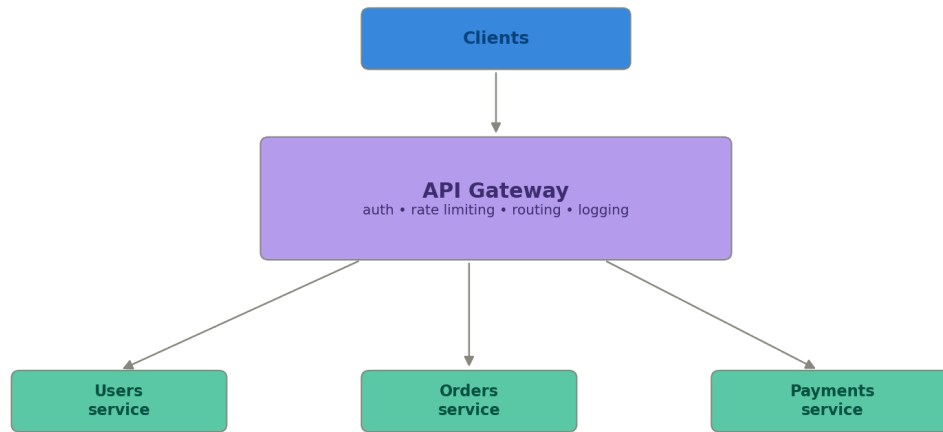


- Token bucket: a bucket holds tokens that refill at a fixed rate; each request consumes one. Allows bursts up to the bucket's capacity, as long as average usage stays under the refill rate — the most common choice because it matches how real traffic actually behaves (bursty, not perfectly smooth).
- Leaky bucket: incoming requests queue up and are processed (“leak out”) at a strictly constant rate. Smooths bursts into steady output, but excess requests wait in the queue rather than being handled immediately — adds latency that token bucket doesn't.
- Sliding window: counts actual timestamped requests within a rolling time window (e.g., the last 60 seconds, continuously). Most accurate at enforcing a true rate, but costs more to implement since it needs to store individual request timestamps rather than just a counter.

Why this is a systems problem, not just an algorithm: a rate limiter implemented as an in-process counter breaks the moment you have more than one server, since each server would track its own count independently — a client could get N times the intended limit by spreading requests across N servers. A real rate limiter needs shared state (typically Redis, using INCR plus a TTL) so every server enforces the same count.

2. API Gateways

An API gateway sits in front of a system's services and centralizes the concerns every service would otherwise have to reimplement on its own.



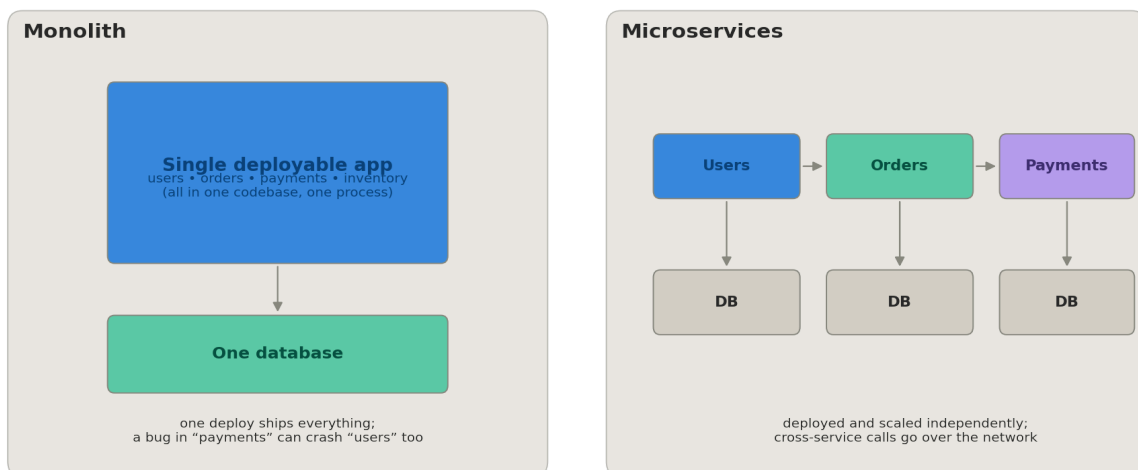
every cross-cutting concern lives in one place,
instead of being reimplemented in each service

- Authentication/authorization: verify who's calling once, at the edge, instead of in every service.
- Rate limiting: enforced in one place, consistently, rather than per-service (and per-server) drift.
- Routing: maps external API paths to the correct internal service, so clients don't need to know your internal topology.
- Logging/observability: a single point to capture metrics and traces for every request entering the system (ties directly into Stage 7).

The tradeoff: the gateway becomes a critical dependency and a potential bottleneck/single point of failure — which is exactly why it's typically deployed as a redundant, horizontally-scaled layer itself (Stage 1's horizontal scaling, applied to the gateway).

3. Microservices vs. Monoliths

This is a real operational tradeoff, not a matter of which is more modern.



- Monolith: one codebase, one deployable unit, typically one database. Simple to develop, test, and deploy early on — a function call is just a function call, with no network involved. The cost: everything scales and deploys together, and a bug in one part can take down the whole process.
- Microservices: independently deployable, independently scalable services, often with their own databases. Lets teams and services scale and ship on their own schedule. The cost: what used to be a function call is now a network call — you inherit every problem from Stage 5 (partial failure, retries, consistency across services) that a monolith simply didn't have.

Decision framing: start with a monolith unless you already know you need independent scaling or independent deployment for specific parts of the system — microservices are a cost paid for a specific organizational or scaling benefit, not a default best practice. Most real systems described in interviews, and most real companies, begin as a monolith and split out services only where a concrete bottleneck or team boundary justifies the operational cost.

4. Service Discovery, Versioning, and Backward Compatibility

- Service discovery: in a world of many service instances that scale up/down dynamically, something needs to track “which IPs are currently valid instances of the Orders service” so callers (or the gateway/load balancer) don't hardcode addresses that can change at any time.
- API versioning: once an API has external consumers, you can't just change its shape — versioning (e.g., /v1/, /v2/) lets you evolve the API while existing clients keep working against the old version.
- Backward compatibility: the practical rule is additive changes are safe (new optional fields), removing or renaming fields is not — it silently breaks any client still expecting the old shape.

Why this matters more in microservices: in a monolith, “changing an interface” is a compile-time error caught immediately. Across services, an incompatible change is a runtime failure discovered in production, often in a service you don't own — which is exactly why versioning discipline substitutes for the safety a monolith gets for free.

How this stage connects back

- Rate limiting at scale needs the shared, distributed state problem from Stage 5 (a single Redis counter is itself a tiny version of “multiple nodes agreeing on one value”).
- The API gateway is Stage 2's load balancer concept, extended from “route traffic” to “route traffic and enforce policy.”
- Choosing microservices is explicitly choosing to take on Stage 5's distributed-systems tax in exchange for independent scaling and deployment — it's not a free upgrade from “monolith.”

Must be able to explain before moving on

1. Why an in-memory rate limiter breaks the moment you have more than one server, and what fixes it.
2. One concrete cost microservices introduce that a monolith doesn't have to deal with at all.
3. Why removing a field from an API response is a backward-incompatible change even if no client code currently reads that field.
4. What specifically an API gateway centralizes, and why that's better than each service implementing its own auth and rate limiting.